类别	2025 年“华数杯”全国大学生数学建模竞赛	参赛编号
本科组		CM2503945

基于可调控生物节律的 LED 光源优化设计与睡眠质量影响分析

摘要

随着健康照明理念的普及，兼顾视觉舒适性与生物节律友好的光源设计成为研究热点。LED 光源因其光谱可调特性，为实现特定场景下的节律调控提供了技术基础。然而，如何在满足色温与显色等视觉指标的同时，有效控制昼夜节律刺激，并通过实验验证其对睡眠质量的影响，仍需系统性研究。为此，本文围绕标准化参数、多通道光谱合成、多目标优化、睡眠指标的统计验证四个环节展开。

在问题一中，建立光源多指标评价体系，包括相关色温（CCT）、显色指数（Rf）、色域指数（Rg）及昼夜节律刺激量（mel-DER），并比较多种计算方法的精度与适用性。计算结果表明，所选光源 CCT 约为 3900 K、色偏极小、显色性优良且 mel-DER 极低，适合夜间使用。

在问题二中，针对日间与夜间两类应用场景进行多通道光谱优化。日间模式以显色性与节律安全性为目标，差分进化算法得到 Rf 接近 100、色域平衡的配置；夜间模式以 mel-DER 最小化为核心，SLSQP_Precise 模型在显色性与光色纯净度间实现最佳平衡，显著降低蓝光成分。

在问题三中，设计模拟全天自然光变化的多目标时序优化方法，引入分时段加权与时序平滑约束，采用两阶段 L-BFGS-B 策略，使合成光谱在节律关键指标上与太阳光谱高度匹配（相关系数超 0.99），并保持全天变化的平滑过渡。该方法在 75 维决策变量下仍能快速收敛，内存占用低且支持边界约束；优化结果在早、中、晚各时段的 CCT 和 mel-DER 误差均处于较低水平，充分验证了模型的精度与稳定性。

在问题四中，利用 11 名被试的多夜睡眠实验数据，对三种光照条件（助眠模式、普通 LED、黑暗环境）进行统计分析。结果显示，助眠模式在部分指标有改善趋势，但深睡眠比例显著低于黑暗环境，表面夜间光谱优化需在控制节律刺激与维持深睡眠质量之间取得平衡。

综上，本文提出了从光谱参数计算、场景化光谱设计到睡眠实验验证的完整技术路径，为生物节律友好型照明设计与健康效应评估提供了可行的模型与方法参考。

关键字：LED 光源优化　生物节律　多通道光谱合成　多目标优化　睡眠质量

一、 问题提出

1.1 问题背景

随着 LED 光源在照明领域的广泛应用，LED 光源高效、节能、环保的特性使其已成为主流光源。然而，除了为人类提供视觉照明，光照的变化对人体的生理节律也有深远的影响。研究发现，不同波长的光照对褪黑素分泌有显著影响，从而调节人体的昼夜节律。如何在满足照明需求的同时，优化 LED 光源的光谱特性，以实现有益的生理节律调节，成为当前照明领域亟待解决的重要问题。

1.2 问题重述

在以上的问题背景下，结合附录数据与参考文献，建立数学模型解决以下问题：

- 问题一：基于给定的 SPD 数据并阅读参考文献，我们需计算出相关色温（CCT）、距离普朗克轨迹的距离（Duv）、保真度指数（Rf）、色域指数（Rg）和褪黑素日光照度比（mel-DER）这五个关键参数，并通过标准化的计算方法和数学模型对比分析不同算法的优缺点。
- 问题二：涉及多通道光源的设计，研究如何通过调整各通道的权重组合，合成适应日间和夜间不同照明需求的光谱，并优化光源的保真度、色温和生理节律效应
- 问题三: 则要求通过控制策略，模拟一天中的自然光变化，借助 LED 光源实现类似的生理节律效应，并进行对比分析
- 问题四：要求结合临床睡眠实验数据，通过统计分析评估光照模式对睡眠质量的影响，验证所设计光源的有效性。

二、 问题分析

2.1 问题一

针对相关色温 (CCT) 和距离普朗克轨迹的距离 (Duv)，我们首先分别利用三角垂足插值法，Chebyshev 法，黑体轨迹弧线模拟法，McCamy 近似公式法求解相关色温，分析出精度较高，较为便捷的算法，而对于距离普朗克轨迹的距离我们选择在 $|Duv|<0.002$ 时使用三角形法，在 $|Duv|\geq 0.002$ 时分别使用三次样条插值法和使用带有二阶导数的三次多项式法。颜色还原参数 (Rf 和 Rg) 则是在 CAM02 - UCS 坐标系统上依据 IES TM-30-20 标准计算，最后 mel-DER 按照 CIE S 026/e:2018 标准计算。

2.2 问题二

针对最优的权重组合，我们设计了“双策略法”优化框架。首先采用 SLSQP 算法快速求得可行的局部最优解，以满足色温、显色性、色域指数和节律刺激等约束；随后引入 SLSQP_Precise、SLSQP_Robust、L-BFGS-B、Differential_Evolution（以下简称 DE）、Basin Hopping 和 Dual Annealing 六种互补优化算法进行并行对比，兼顾局部精细搜索与全局探索能力。该方法既利用梯度类算法在连续可微空间的高效性，又借助随机全局算法提升跳出局部极值的能力，为日间与夜间两种场景下的 LED 光谱设计提供鲁棒且高性能的权重组合方案。

2.3 问题三

先通过波长解析与标准化、时间范围筛选（聚焦 05:30-19:30 有效时段）、太阳光谱矩阵构建、LED 通道光谱提取及光谱特征统计完成数据预处理；核心优化采用 L-BFGS-B 算法，将 LED 光谱合成建模为带边界约束的多目标优化问题，目标函数结合分时段加权的多指标（CCT、Duv 等五项）拟合项与时序平滑项，以追踪太阳光谱指标轨迹并保证权重序列平滑；同时设计两阶段策略，阶段 A 分时段全局拟合核心节律指标，阶段 B 引入显色性指标并通过稀疏计算提升效率；结果显示合成光谱在五项关键指标上与太阳光谱高度匹配，CCT 和 mel-DER 的皮尔逊相关系数超 0.99，早中晚关键时段的 RMSE 较低，验证了该方法在动态模拟太阳光谱及兼顾生物节律与视觉品质上的有效性。

2.4 问题四

问题四旨在利用临床睡眠实验数据，评估不同光照条件对睡眠质量的影响，从而验证前期光谱优化的效果。实验采用 11 名健康被试的交叉设计，每位被试分别在夜间助眠模式、普通 LED 灯光和严格黑暗三种条件下过夜，睡眠阶段数据按 30 秒记录并遵循 AASM 标准。首先数据预处理，统一格式并剔除异常值；然后计算总睡眠时间、睡眠效率、入睡潜伏期、深睡眠比例、REM 比例和夜间醒来次数六项指标；最后进行正态性和方差齐性检验，满足条件时采用重复测量方差分析及事后配对 t 检验，并结合效应量评价实际意义。

图 1　方案流程图

三、 模型假设

1. 光源的光谱功率分布（SPD）在测量或给定范围内稳定，不随时间波动。
2. 各 LED 通道光谱功率之间线性可叠加，且通道间相互独立，权重调整不影响其他通道的光谱形状。
3. CCT、Duv、Rf、Rg 与 mel-DER 的计算均符合 CIE、IES TM-30-20 等国际标准，计算过程不引入系统性误差。
4. 光照对人体生理节律的影响主要通过 mel-DER 反映，忽略其他次要光生物学效应。
5. 睡眠实验中，除光照条件外，被试的睡眠环境（温度、噪声、作息等）保持一致，且不同实验夜之间不存在显著的外部干扰因素。
6. 睡眠监测仪器测量准确可靠，数据缺失或异常已通过预处理剔除，不影响后续统计分析。
7. 不同被试的个体差异服从随机分布，交叉实验设计可有效消除个体差异的系统性影响。

四、 符号说明

符号	意义
λ	波长（nm）
$SPD(\lambda)$	光谱功率分布（Spectral Power Distribution, W/nm）
CCT	相关色温（Correlated Color Temperature, K）
D_{uv}	色度点与普朗克轨迹的距离（色偏）
R_f	保真度指数（Fidelity Index）
R_g	色域指数（Gamut Index）
mel-DER	褪黑素日光照度比（Melanopic Daylight Efficacy Ratio）
$P_i(\lambda)$	第 i 个 LED 通道的相对光谱功率分布
w_i	第 i 个 LED 通道的权重系数
$SPD_{mix}(\lambda)$	多通道加权合成光谱
TST	总睡眠时间（Total Sleep Time, min）
SE	睡眠效率（Sleep Efficiency, %）
SOL	入睡潜伏期（Sleep Onset Latency, min）
$N3\%$	深睡眠比例（N3 阶段时间占比）
$REM\%$	REM 睡眠比例
NAW	夜间醒来次数（Number of Awakenings）

五、 问题一: 标准化参数及计算

5.1 相关色温 (CCT)

根据所提供的文献，我们使用文献中的 McCamy 法，Chebyshev 法，三角垂足插值法和模拟黑体轨迹弧线法 [5] 共计四种方法分别进行计算，将结果对比分析后确定参数值。从所给文献与实际建模中我们对四种方法的优缺点进行了分析:

- McCamy 法: 能够快速计算 CCT，广泛用于低色温光源的初步评估。通过色度坐标计

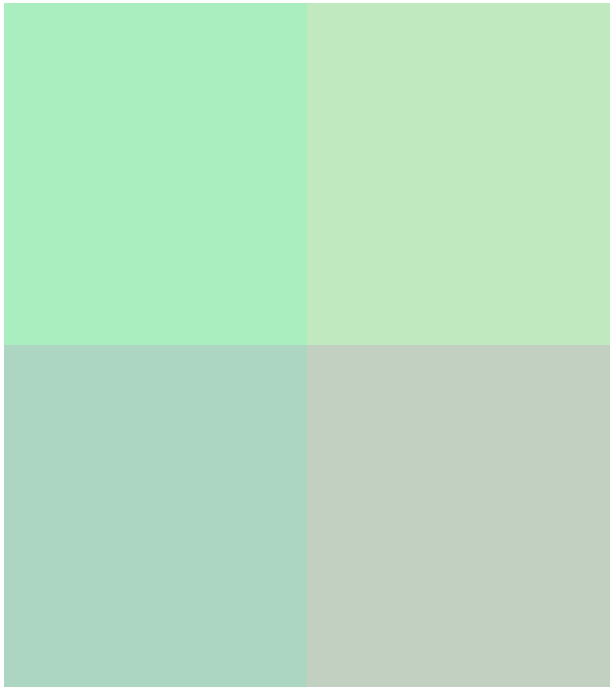


图 2　CIE 1931 XYZ 色彩空间

算 CCT，计算过程简洁高效，但由于公式是通过经验拟合得到的，对于高色温光源的计算精度较低。

- Chebyshev 法：通过 Chebyshev 多项式拟合色度坐标与 CCT 之间的关系，提供了较高的计算精度，适用于低色温光源。利用最小化目标函数，Chebyshev 法可以优化 CCT 的计算精度。
- 三角垂足插值法：通过计算光源色度坐标与黑体轨迹的最短距离来确定 CCT，对高色温光源的计算尤为有效，能够提供非常精确的结果。
- 模拟黑体轨迹弧线法：通过模拟黑体辐射轨迹的弧线，快速计算 CCT 值。适用于较大色温范围的光源（如从 3000K 到 10000K 的范围），并且计算效率较高。

计算 CCT 参数步骤如下：

Step 1: 将附录中的 SPD 数据转换为在 CIE 1931 XYZ 色彩空间 (如图 2所示) 中的坐标，计算 CIE XYZ 三刺激值，作为四种方法计算的前提。为了使计算更加精确，我们使用了标准的 CIE 匹配函数并对 SPD 数据进行了插值处理。通过数值积分，计算出最终的三刺激值。公式如下：

$$X = \int_{380}^{780} S(\lambda)\bar{x}(\lambda)d\lambda, \tag{1}$$

$$Y = \int_{380}^{780} S(\lambda)\bar{y}(\lambda)d\lambda, \tag{2}$$

$$Z = \int_{380}^{780} S(\lambda)\bar{z}(\lambda)d\lambda, \tag{3}$$

其中，$S(\lambda)$ 为光源的 SPD 数据，$\bar{x}(\lambda)$ 、$\bar{y}(\lambda)$ 、$\bar{z}(\lambda)$ 为 CIE 1931 标准色度观察者的色匹配函数。

Step 2: 针对已计算出的三刺激值进行归一化, 从而计算色度坐标：

$$x = \frac{X}{X+Y+Z}, \tag{4}$$

$$y = \frac{Y}{X+Y+Z}. \tag{5}$$

基于 (*x*,*y*) 坐标，分别使用三角垂足插值法，Chebyshev 法，黑体轨迹弧线模拟法，McCamy 近似公式法计算 CCT, 结果如下。

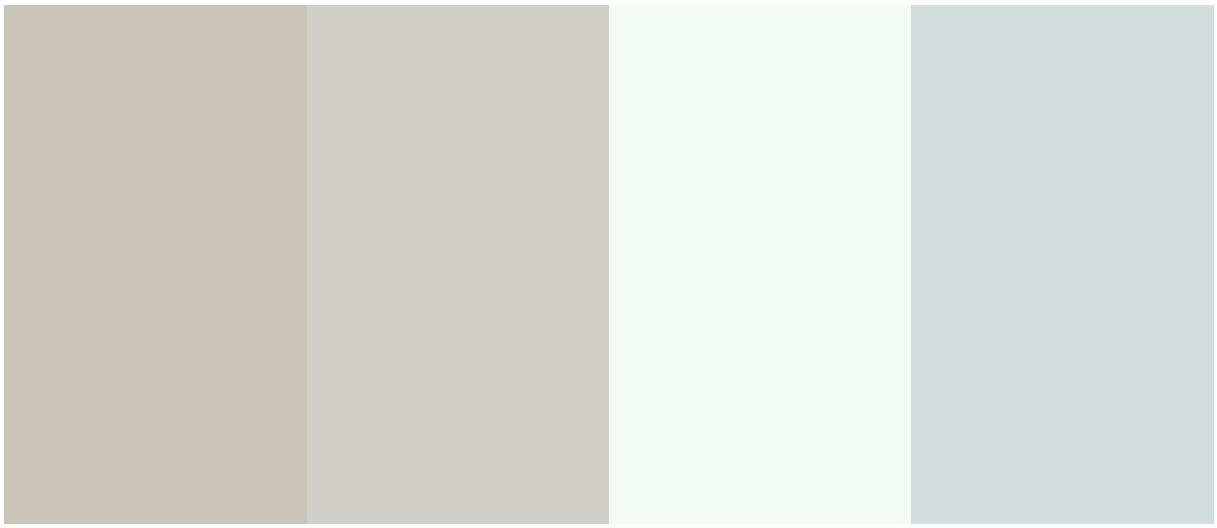


图 3　CCT 结果及运算速度对比

从图 3中，我们可以显然看出左侧 “CCT Results Comparison” 中，Chebyshev 法的相关色温 (CCT) 值高达 5914K，远高于 McCamy 法（3903K）、三角垂足插值法（3903K）和模拟黑体轨迹弧线法（3902K), 后三者数值相近; 在右侧 “Computation Time Comparison” 中，Triangle 计算时间为 0.003843 秒，远长于 McCamy、Chebyshev、模拟黑体轨迹弧线法（均小于 0.000001 秒），可见不同方法在色温结果和计算效率上存在差异。Chebyshev 法色温表现突出，Triangle 法计算耗时多。

在本研究中，我们的目标是设计既适用于日常照明，又能有效调节人体生物节律的 LED 光源。一般来说，CCT 在 3000K 到 4000K 之间的光源适合家庭和办公照明，同时有助于保持生物节律。而 5000K 以上的 CCT 通常表示冷色光源，这种光源显然对于人类生物节律调节的效果较差，可能影响褪黑素的分泌，尤其在夜间使用时可能对健康产生负面影响。尽管 Chebyshev 法能够提供较高的精度，但其计算结果在本实验中的偏

差过大。5000K 以上的 CCT 值显然不符合我们的设计目标，因此我们决定忽略这一结果，优先使用其他方法计算的 CCT 值。虽然模拟黑体轨迹弧线法在数值上给出的目标略低于 **McCamy 法**和**三角垂足插值法**，但该差异处于数值计算的容差范围内, 考虑到 McCamy 法和三角垂足插值法在不同求解机制下均得到一致的结果，且该结果在约束可行性与收敛稳定性方面表现更优，因此终推荐的 CCT 值为 **3903K**。

5.2 距离普朗克轨迹的距离 (Duv)

在光源显色性分析中，Duv 值是衡量光源色偏的重要指标，特别是对于那些色品坐标偏离黑体轨迹的光源。根据所提供的文献内容，距离普朗克轨迹的距离（Duv）值可以通过 CIE 1960 UCS 颜色空间中的色度坐标与黑体轨迹之间的距离来计算。为了高精度的计算 Duv，对于属于不同值域的 Duv，我们采用不同的方法。

5.2.1 初步计算

首先根据相关色温时计算出的（x,y）坐标转换为 (u, v) 坐标，式子如下：

$$u = \frac{4x}{-2x + 12y + 3}, \tag{6}$$

$$v = \frac{6y}{-2x + 12y + 3}, \tag{7}$$

进一步进行 Duv 的计算，公式如下：

$$\mathrm{Duv} = \sqrt{(u_\mathrm{t} - u_\mathrm{bb})^2 + (v_\mathrm{t} - v_\mathrm{bb})^2}, \tag{8}$$

其中，u_t 和 v_t 为测试光源的色度坐标，而 u_bb 和 v_bb 为参考黑体光源的色度坐标。

5.2.2 分值域计算

- **三角形法**：当 $|\mathrm{Duv}| < 0.002$ 时，采用三角形法进行计算。这是基于几何关系的方法，计算简单，适用于小的色偏。
- **三次样条插值法 [7]**：当 $|\mathrm{Duv}| \geq 0.002$ 时，采用三次样条插值法进行优化。通过拟合样本数据的三次样条，减少误差，提高计算精度。
- **带有二阶导数的三次多项式法**：也适用于 $|\mathrm{Duv}| \geq 0.002$，通过高阶多项式拟合进一步提升精度。

基于以上方法，计算结果如下：

表 1　算法结果

Triangle	相关色温	Duv 值	到黑体轨迹距离
结果	3903.1 K	-0.001003	0.001003

结果表现出色温为 3903K，处于 2700K-4000K 的暖白光范围，适用于住宅照明、餐厅照明及氛围营造等场景；其 Duv 值为 - 0.001003，呈现偏品红色特征，绝对值 |Duv|=**0.001003**，依据 | Duv|<0.002 的评价标准，该光源的此参数表现优秀。在光源品质方面，该光源具有高精度的色温表现，计算误差小于 1K，色彩还原效果接近理想黑体。

5.3 颜色还原参数 (Rf 和 Rg)

在光源显色性评估中，保真度指数（Rf）和色域指数（Rg）是两个关键指标，它们衡量了光源的颜色还原能力和色域表现。根据 IES TM-30-20 标准，通过与参考光源（通常为标准白光光源）对比，基于 99 个测试颜色样本（TCS）数据 [3] 进行计算。根据所提供文献的研究方法，我们详细分析了测试光源的显色性表现，并得出了 Rf 和 Rg 的计算结果。

5.3.1 保真度指数（Rf）计算方法

根据 Royer et al. (2022) 的方法，Rf 是通过计算每个 TCS 样本在测试光源与参考光源下的色度坐标差异来获得的。保真度指数反映了测试光源与参考光源之间的色差。我们采用以下步骤 [8] 计算 Rf 在 CAM02 - UCS 颜色空间，色差通过欧几里得距离计算得出，公式如下:

$$\Delta E_{Jab,i} = \sqrt{(J'_{t,i} - J'_{r,i})^2 + (a'_{t,i} - a'_{r,i})^2 + (b'_{t,i} - b'_{r,i})^2}, \tag{9}$$

其中，$(a_{t,i}$，$b_{t,i})$,$(a_{r,i}$,$b_{r,i})$ 为标准色样分别在待测光源和参照光源下 CAM02 - UCS 颜色空间的色坐标。根据以上色差参数，色保真指数计算公式如下:

$$R'_{\mathrm{f}} = 100 - 7.54\left(\frac{1}{99}\sum_{i=1}^{99}\Delta E_{Jab,i}\right), \tag{10}$$

最终，保真度指数 Rf 的计算公式为:

$$R_{\mathrm{f}} = 10\ln\left(\mathrm{e}^{\frac{R'_{\mathrm{f}}}{10}} + 1\right). \tag{11}$$

5.3.2 色域指数（Rg）计算方法

色域指数提供的是一种被测试光源相对于参照光源所做的色域测量值。其意义在于提供一个评价被不正确呈现的颜色是否过饱和或欠饱和的新度量标准。Rg 是由标准颜色样品在待测光源和参考光源的在 CAM02 - UCS 坐标系统 16 个分区中色坐标 (a' , b') 平均值连成的多边形面积的比值。具体计算公式如下:

$$R_g = 100 \times \frac{A_t}{A_r}, \tag{12}$$

其中，A_t，A_r 分别代表标准颜色样品在待测光源和参考光源下的在 CAM02 - UCS 坐标系统形成的多边形面积值。当 Rg 为 100，表示被测试光源能如同参照光源具有相同的平均饱和度水平呈现色彩。Rg 大于 100 指示过饱和，而低于 100 则显示欠饱和，Rg 的取值取决于应用环境。

5.3.3 Rf 与 Rg 结果分析

图 4　16 色相区域分析极坐标图

图 4 展示了测试光源的保真度指数（Rf）与色域指数（Rg）在 16 色相区域的表现情况。极坐标图中，蓝色区域代表 Rf，橙色环带代表 Rg。结果显示，光源的总体 Rf 为 **87.4**，总体 Rg 为 **128.3**。

从图中可以看出，该光源在 H1 区域（红色）的表现尤为突出，Rf 达到 100，表明其在红色的颜色还原方面完全匹配参考光源；与此同时，Rg 在该区域显著高于 100，说明饱和度有所提升。在 H9 区域（青色），Rf = 87.1，Rg = 96，表现稳定，饱和度略低于参考光源，但仍保持较高的色彩还原性。其余部分多数色相区域的 Rg 值超过 100，反映出整体色域扩展性较强，尤其在暖色调方向更为明显。

综合来看，该光源在红色区域既保持了极高的保真度，又具备明显的饱和度增强；在大部分色相区间均能呈现高于参考光源的色域表现，符合 P-V1 F2 分类标准的高性能照明要求，能够带来鲜艳且真实的色彩呈现效果。

5.4 褪黑素日光照度比 (mel-DER)

在光源设计与评价中，光源对人体昼夜节律的影响是一个重要的考虑因素。根据 CIE S 026/e:2018 标准，褪黑素日光照度比（mel-DER）是用于量化光源对人体昼夜节律影响的关键指标之一。该指标根据光源的光谱功率分布（SPD）和褪黑素敏感度函数（S_{mel}）[4] 进行计算，衡量光源对褪黑素分泌的潜在影响。因此通过计算测试光源的 mel-DER 值，评估其对昼夜节律的刺激效果。具体计算过程如下：

Step 1：加载并预处理光谱功率分布（SPD）数据，提取对应的波长及其光谱功率值，清除异常或无效数据，并对光谱功率进行归一化处理。

Step 2：获取褪黑素光谱敏感度函数（S_{mel}），用于反映视网膜内的视网膜节细胞（ipRGCs）对不同波长光的相对敏感性；同时获取明视觉光谱效率函数（V_λ）。

Step 3：通过积分计算 mel-opic 辐照度（即 SPD 与 S_{mel} 的乘积积分）以及光通量（即 SPD 与 V_λ 的乘积积分乘以最大光视效率常数 K_m，约为 683 lm/W）。具体计算公式如下：

$$\gamma_{\mathrm{mel,v}}^{\mathrm{D65}} = \frac{K_{\mathrm{mel,v}}}{K_{\mathrm{mel,v}}^{\mathrm{D65}}} = \frac{\Phi_{\mathrm{mel}}/\Phi_{\mathrm{v}}}{\Phi_{\mathrm{mel}}^{\mathrm{D65}}/\Phi_{\mathrm{v}}^{\mathrm{D65}}}, \tag{13}$$

其中，$K_{\mathrm{mel,v}}^{\mathrm{D65}}$ 表示 D65 标准日光的褪黑素光效，为固定值 $1.3262 mW \cdot lm^{-1}$，$K_{\mathrm{mel,v}}$ 是光源的褪黑素光效（melanopic efficacy of luminous radiation），表示单位光通量（lm）对应的褪黑素有效辐射通量（mW），计算方式如下：

$$K_{\mathrm{mel,v}} = \frac{\int \Phi(\lambda) \cdot S(\lambda)\, d\lambda}{\int \Phi(\lambda)\, d\lambda}, \tag{14}$$

其中，$\Phi(\lambda)$ 表示光源的光谱辐射通量，$S(\lambda)$ 表示褪黑素作用光谱。

Step 4：计算 mel-opic 辐照度与光通量的比值，即 mel-opic ELR，并以 D65 标准光源的 ELR 作为参考基准，得到目标光源相对于 D65 的 mel-DER 值。

Step 5：基于 mel-DER 计算 mel-opic 等效日光照度（mel-opic EDI），并根据其数值对昼夜节律刺激强度进行分级，划分为高、中、低及极低等级。

表 2　算法结果

参数	数值	评价
mel-DER	**0.001**	极低昼夜节律刺激
mel-opic EDI	22.9 lux	——
D65 光源（参考）	ELR=1.326	——
建议使用时间	22:00 –06:00	睡前或夜间使用

六、 问题二：最优通道权重组合

6.1 预处理与总体策略

6.1.1 统一计算框架与数据处理

为满足 IES TM - 30 标准要求，需对光谱功率分布（SPD）数据进行预处理操作。首先，依据标准规定，将波长网格统一调整为 5nm 步长，该步长下共包含 81 个采样点，以此构建规范且一致的波长采样框架 [6] 。接着，采用线性插值方法，针对各通道的 SPD 数据，在新的统一波长网格上进行数值插补，使不同通道数据在相同波长间隔下具备可比性。随后，为保障数值计算过程中的稳定性，对各通道数据开展归一化处理，将每个通道 SPD 数据的最大值缩放至 1，消除量纲及数值差异带来的影响。同时，考虑到光谱功率分布的物理意义，对处理后的数据进行非负性约束，严格限制 SPD 值处于大于等于 0 的范围，确保数据符合实际物理规律，为后续基于 TM - 30 标准的颜色质量分析等操作奠定可靠的数据基础。

在上一问的基础上，我们先分别采用三角垂足插值法、三角形法和三次样条插值法对多通道对多通道 LED 光谱进行重构，以保证波长采样的连续性和平滑性；随后依据 IES TM-30-20 标准计算 Rf 与 Rg，并结合 CIE S 026/E:2018 标准计算 mel-DER，最终获得不同插值方法下的光谱特征参数值。

6.1.2 总体策略

针对本题涉及的多通道光谱权重优化问题，我们在建模初期规划了两类优化策略，以兼顾不同复杂度和应用场景的需求。

策略 A： 以 SLSQP 算法 [2] 为核心，利用其在非线性约束处理和梯度收敛精度方

面的优势，在已知解空间较为平滑、约束条件稳定的情况下快速收敛，计算效率高，结果精度优良，适合对单一性能指标（如 Rf,CCT）进行精细调优。

策略 B：采用多算法并行对比方案，适用于高维度、多目标及复杂约束的情形。在统一的光谱预处理和物理约束条件下，我们使用了六种代表性算法，具体如下：

- **SLSQP_Precise（高精度局部优化）**：通过收紧收敛阈值、提升梯度估计精度，实现对 Rf、RG 及 Duv 的细致调节。在 SPD 合成和 TM-30 计算中，能快速锁定精细化平衡解，适合在可行域边界时进行参数的微调。
- **SLSQP_Robust（鲁棒性局部优化）**：在保持 SLSQP 基础框架的同时放宽梯度精度要求，并增加对不稳定解的重启机制。适于光谱合成后数值波动较大的情形，在 CCT 和 Duv 存在扰动时保持稳定收敛。
- **L-BFGS-B（拟牛顿法）**：在目标函数中显式加入对越界权重和指标偏差的惩罚项，在每轮 SPD 计算后立即检查约束并调整梯度方向。
- **DE（差分进化，全局）**：通过种群进化机制在全局范围搜索，天然具备跳出局部最优的能力。我们设置了较宽的 CCT–Duv 初值分布，并在每代进化中保留显色性较优的个体，以加速收敛到高 Rf 解。
- **Basin_Hopping（盆地跳跃，全局）**：将随机扰动与局部优化结合，在权重空间形成跨越“盆地”的搜索路径，可在多个 SPD 局部极值之间跳转，有助于发现 CCT 和 RG 的更优平衡的解。
- **Dual_Annealing（双退火，全局）**：模拟退火与局部搜索的结合，能够逐步降低搜索温度以锁定稳定解。适用于 mel-DER 与显色性存在冲突时，通过温度控制在早期探索全局、后期精细收敛。

策略 A 利用单一高效的局部优化方法，能够在短时间内给出满足基本约束的可行解，为后续探索提供性能基准与参数参考；策略 B 则引入多种全局与局部相结合的优化算法，显著提升了跳出局部最优、在多指标间实现全局平衡的能力。在场景一二下，我们均对两个策略展开了尝试。

6.2 场景一: 日间照明模式

在场景一中，我们的目标是在模拟相比夜间模式，日间模式更强调色彩还原度与整体视觉舒适度。因此在优化过程中会优先兼顾显色性与色度表现，在满足日间照明需求的同时兼顾节律效应的合理性。

6.2.1 设定限制条件

根据题目条件有以下约束：

色温约束：合成光谱的相关色温 CCT 必须位于日光范围内，即 $5500K \le CCT \le 6500$ K；

显色指数约束：$R_f(w) \ge 88$，以保证基本显色性能；

色域指数约束：$95 \le R_g(w) \le 105$ ，避免色彩过饱和或欠饱和；

权重约束：通道权重非负且归一化，即 $w_i \ge 0$（i=1,2,3,4,5 ）且 $\Sigma w_i = 1$ 。

6.2.2 模型的比较与选择

在研究的初期阶段，我们首先采用了单一的顺序最小二乘（SLS）方法作为策略 A，对其在本问题中的适用性进行探索。然而，由于该问题的目标函数呈现出多峰且结构复杂的特性，SLS 在搜索过程中容易停留在局部最优点，难以在 Rf 与 mel-DER 之间实现理想的权衡。基于此局限性，我们进一步构建了策略 B——多算法并行优化框架 [1]，旨在结合不同优化算法的全局探索能力与局部精细化搜索优势，从而显著提升最终解的整体质量。

在六种算法的运算下，我们发现以下四种均存在缺陷：SLSQP_Precise 虽在局部收敛精度上表现优异，但在本问题的多峰目标函数下容易陷入局部最优，导致 Rf 与 Rg 无法同时满足约束；L-BFGS-B 在连续性和平滑性优化上具备优势，但在色温与显色性平衡上收敛不足；Basin_Hopping 与 Dual_Annealing 具备全局搜索能力，但在收敛速度和最终精度上不如最优的两种，且多次运行结果波动较大。综合考虑稳定性、精度与计算效率，我们最终选用 **DE** 与 **SLSQP_Robust** 模型。

6.2.3 DE 模型建立

DE 模型是一种基于种群的优化算法，特别适用于全局优化问题。它通过模拟自然进化过程中的变异、交叉和选择来寻找最优解。以下是我们使用 DE 算法的详细步骤：

Step 1：在每个通道权重的可行范围内，随机生成若干组候选解，每组解表示一个通道权重组合。这些组合构成了初始种群，确保算法从多样化的起点出发，覆盖广泛的搜索空间。

$$\mathbf{x}_{i,0} = \mathbf{x}_{\min} + \mathbf{r}_i(\mathbf{x}_{\max} - \mathbf{x}_{\min}), \tag{15}$$

其中，$\mathbf{r}_i$ 是一个在 $[0,1]$ 区间内均匀分布的随机向量。

Step 2：差分进化算法通过在现有候选解之间引入随机扰动，生成新的试验解。在变异操作中，选取若干个不同个体的差向量进行线性组合，以形成新的搜索方向；随后

在交叉操作中，将变异产生的向量与原个体按一定概率混合，从而增加解的多样性并提升全局探索能力。然后对每个个体 $\mathbf{x}_{i,G}$（第 G 代的第 i 个个体），选择三个互不相同的个体 $\mathbf{x}_{r1,G}$, $\mathbf{x}_{r2,G}$ 和 $\mathbf{x}_{r3,G}$ （且 $r1 \neq r2 \neq r3 \neq i$），从而构建变异向量：

$$\mathbf{v}_{i,G} = \mathbf{x}_{r1,G} + F \cdot (\mathbf{x}_{r2,G} - \mathbf{x}_{r3,G}), \tag{16}$$

其中，F 是变异因子，通常在 [0, 2] 之间。

$$u_{i,j,G} = \begin{cases} v_{i,j,G} & \text{if } rand_j(0,1) \leq CR \text{ or } j = j_{\text{rand}} \\ x_{i,j,G} & \text{otherwise} \end{cases}, \tag{17}$$

其中，$rand_j(0,1)$ 是一个在 [0, 1] 区间内均匀分布的随机数，j_{rand} 是 $[1, D]$ 之间的一个随机索引，确保至少有一个维度来自变异向量。

Step 3：对于每一个新生成的候选解，先根据权重系数对五个通道的 SPD 进行加权叠加，得到合成光谱。然后，利用标准化计算方法获得该光谱的关键性能参数，包括相关色温（CCT）、距离普朗克轨迹的偏移量（Duv）、保真度指数（Rf）、色域指数（Rg）以及褪黑素日光照度比（mel-DER）。

$$\mathbf{x}_{i,G+1} = \begin{cases} \mathbf{u}_{i,G} & \text{if } f(\mathbf{u}_{i,G}) \leq f(\mathbf{x}_{i,G}) \\ \mathbf{x}_{i,G} & \text{otherwise} \end{cases}. \tag{18}$$

Step 4：通过比较新候选解与原解在目标函数上的表现，保留表现更优的解进入下一代种群。该过程反复迭代，逐步逼近全局最优解。

Step 5：在算法终止后，输出性能最优的通道权重组合及其对应的光谱参数。

6.2.4 SLSQP_Robust 模型建立

SLSQP_Robust 模型能够通过精确的梯度与 Hessian 信息，引导优化过程在保证可行解的前提下快速逼近全局最优解。以下是我们使用 SLSQP_Robust 算法的步骤：

Step 1：在 SLSQP_Robust 框架下，计算目标函数和约束条件的梯度信息。目标函数根据不同场景进行设定，例如在日间模式下以提升显色性指标（Rf）为主，夜间模式则以降低 mel-DER 为核心。约束条件涵盖色温范围、显色性下限、色域指数区间等，这些均由通道权重合成光谱后计算得到。梯度信息可通过数值差分或解析方法获得，为后续二次规划子问题提供必要输入。在 SLSQP_Robust 框架中，首先需要获得目标函数和约束条件的梯度信息。

Step 2：在处理等式和不等式约束时，引入拉格朗日乘子，将目标函数与约束条件统一到一个扩展的拉格朗日框架中。这一构造方式使得约束的处理更加系统化，便于同时考虑多种性能指标与物理限制。

Step 3：在当前迭代点附近，对所有约束条件进行一阶泰勒展开，从而将原本的非线性约束近似为线性形式。这一近似降低了子问题的复杂度，使得在每一步迭代中可以用高效的凸优化方法求解。

Step 4：在线性化约束的前提下，构造一个以目标函数梯度和 Hessian 近似矩阵为基础的二次优化模型。求解该子问题可以得到当前迭代方向，该方向既考虑了目标函数的下降趋势，也兼顾了约束的可行性。

Step 5：基于迭代方向，通过线搜索确定合适的步长因子，以确保更新后的通道权重不仅能够改善目标函数值，还能维持在约束的可行域内。随后更新权重向量，进入下一轮迭代。

Step 6：当迭代达到预设收敛标准（如梯度范数足够小或目标函数改进低于阈值）时，算法终止。输出的最优权重组合即为在给定场景下性能最优的光谱合成方案，同时附带其光谱特征参数。

6.2.5 结果对比分析

在通过 DE 和 SLSQP_Robust 模型计算后，我们分别得到了满足物理与显色性约束的全局最优通道权重组合，各关键指标如下表所示。

表 3　场景一参数结果

指标	CCT	R_f	R_g	Duv	$mel-DER$
SLSQP_Robust	6250	96.9	97.5	-0.0031	0.001
DE	5870	99.9	99.9	-0.0032	0.001

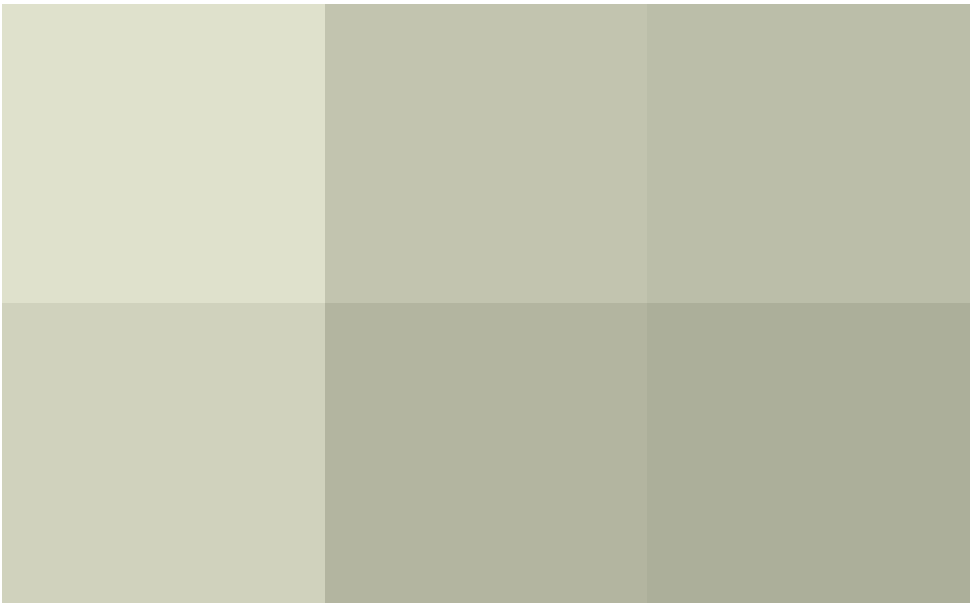


图 5　模型对比图



图 6　模型对比图

从表 3 和图 5图 6可以直观地看出 **DE 模型（差分进化）**显色指数接近 100，该光源下物体颜色的还原度几乎完美；同时在色彩平衡优异上 Rg 接近 100，说明色彩既不过饱和也不欠饱和；由于在照明应用中，显色性能的优势通常比色温的细微差异更重要，实用价值更高；我们认为色温在 5870K 与 6250K 的差异不如显色指数 96.92 与 100 的差异明显，故结论为 **DE 模型（差分进化）**能提供更好的色彩还原效果，在大多数照明场景下都更有价值。

表 4　场景一权重配比

	Blue	Green	Red	Warm White	Cold White
权重	0.2042	0.1494	0.2012	0.0063	0.4390

如上表所示为 DE 模型下计算出的 LED 通道权重配比。

6.3 场景二：夜间助眠模式

在场景二中，我们关注的是在满足夜间照明约束条件下，同时兼顾昼夜节律刺激（mel-DER）和显色性能的光谱权重优化问题。与场景一不同，此处的目标函数更加侧重抑制短波蓝光成分，以降低对人类生物钟的干扰，同时保持足够的显色指数和优质的色度表现。

6.3.1 设定限制条件

根据题目条件有以下约束：

- **色温约束**：$2000K \le CCT \le 3000$，符合夜间用光习惯；
- **色度约束**：色度偏差 $|Duv| \le 0.005$，保证光色纯正无明显色偏；
- **显色性约束**：显色指数 $R_f \ge 80$，在降低蓝光成分的同时维持基本的色彩还原能力；
- **昼夜节律约束**：mel-DER ≤ 0.00035，减轻夜间光照对人体生物钟的干扰；
- **权重约束**：通道权重非负且归一化，即 $w_i \ge 0$（i=1,2,3,4,5）且 $\Sigma w_i = 1$ 。

6.3.2 模型建立

在场景二中，我们完成与场景一相似的数据处理后，我们对策略 A 与策略 B 分别展开尝试，由于策略 A 出现与场景一的相似情况，我们采用策略 B，对前述 6 种方法进行了多轮对比试验。结果显示 Basin_Hopping 与 Dual_Annealing 的迭代次数过多并且精度偏差极大，因此我们选用 L-BFGS-B,SLSQP_Precise,DE,SLSQP_Robust 共四个模型。场景一已经对 DE 模型与 SLSQP_Robust 模型进行过说明，故在此不再赘述，着重说明 L-BFGS-B 模型。

为求解该约束优化问题，我们采用的 L-BFGS-B 算法是一种拟牛顿优化技术，利用历史迭代信息构造 Hessian 矩阵的低秩近似，从而在保持二阶收敛速度的同时显著降低内存需求，适合场景二这种中等维度、连续变量且具有边界约束的场景。

L-BFGS-B 在求解过程中依赖梯度信息进行迭代更新。梯度通过数值差分近似计算：

$$\frac{\partial f}{\partial w_j} \approx \frac{f(\mathbf{w}+\varepsilon \mathbf{e}_j) - f(\mathbf{w}-\varepsilon \mathbf{e}_j)}{2\varepsilon}, \tag{19}$$

其中 $\mathbf{e}_j$ 为单位向量，ε 为差分步长。边界约束由算法内置的投影机制自动处理，使得更新后的 $\mathbf{w}$ 始终位于可行域内。

在我们的实验中，L-BFGS-B 找到的解在显色指数 R_f 和色温精度方面表现优异：所得光谱的 R_f 接近约束上限，且 CCT 稳定落在目标区间中值附近。虽然在 mel-DER 最小化方面略逊于 Differential Evolution 的结果，但其收敛速度更快（计算时间仅为进化算法的约 30%），且多次运行的结果稳定性较高。这使得 L-BFGS-B 在需要快速迭代或实时优化的场景下具有明显优势。

6.3.3 结果分析

表 5　场景二四个表现优异模型的关键指标

模型	$mel-DER$	R_f	R_g	Duv	$CCT(K)$
Differential_Evolution	0.000322	80.0	96.1	0.0025	2480
SLSQP_Robust	0.000324	80.1	95.9	0.0024	2492
SLSQP_Precise	0.000329	86.8	98.0	0.0013	2508
L-BFGS-B	0.000340	86.9	97.8	0.0015	2590

根据表 5 数据可知，**DE 模型**在 mel-DER 控制方面表现最佳（0.000322），能最大限度降低夜间蓝光刺激，但显色性较弱（Rf=80.0），在需要高视觉质量的场景中存在不足。**SLSQP_Robust 模型**的 mel-DER 仅比最优值高 0.6%，Duv 较低，光色稳定，但显色性能与 DE 接近，未带来额外提升。**SLSQP_Precise 模型**在 mel-DER 上虽比最优值高约 2%，但显色性显著提升至 86.8，比前两者高出约 6.7%，且 Duv 为四者最低（0.0013），光色最为纯正，CCT 位于夜间暖光舒适区间，整体平衡最佳。**L-BFGS-B 模型**的显色性最高（Rf=86.9），与 SLSQP_Precise 接近，但 mel-DER 升至 0.000340（较最低值高约 5.6%），且 CCT 略偏高至 2590K。综合五项指标，**SLSQP_Precise 模型**实现了对夜间节律干扰低、显色性高与光色纯净度的最佳平衡，被确定为夜间模式最优解。

图 7与图 8进一步验证了上述结论。**SLSQP_Precise** 与 **L-BFGS-B** 在 Duv 指标上均显著优于 DE 和 SLSQP_Robust，色度偏差更小，更接近目标太阳光谱；在 Rg 指标上保

图 7　模型对比图

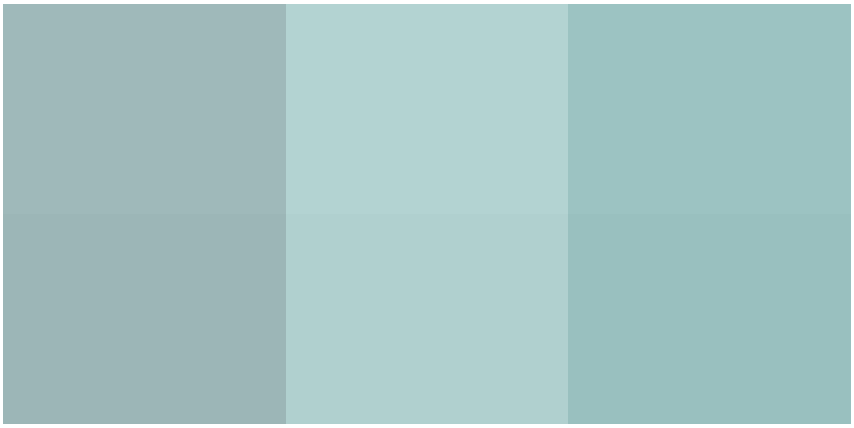


图 8　模型对比图

持在 97.8 ~ 98.0 的较高水平，色域覆盖合理且无明显偏饱和或欠饱和现象；CCT 均与目标设定相符。虽然四种模型在 mel-DER 上差异不大，但综合色度精度、显色性与色域平衡，SLSQP_Precise 与 L-BFGS-B 展现了更优的光谱匹配性能，其中 **L-BFGS-B** 在保持多指标平衡和时序平滑性方面更具优势，适于作为后续节律优化阶段的基础模型。

表 6　场景二权重配比

	Blue	Green	Red	Warm White	Cold White
权重	0.3721	0.0329	0.0066	0.5869	0.0016

表 6 为 SLSQP_Precise 模型下计算出的 LED 通道权重配比。

七、 问题三：模拟太阳光谱的多目标时序优化方法

7.1 光谱数据预处理与矩阵构建

我们先针对 Problem 3 SUN_SPD 的数据进行预处理，步骤如下：

Step 1 波长解析与标准化：通过正则表达式技术对输入数据中的波长信息进行解析，提取纯数值波长，实现不同数据源波长尺度的统一与标准化。

Step 2 时间范围筛选：针对太阳光谱数据中的时间维度，将时间范围限定在了早晨 05:30 至傍晚 19:30，确保分析集中于日照强度和光谱特性显著变化的有效时段。目的是剔除夜间无效观测，增强数据的时序相关性。

Step 3 太阳光谱矩阵构建：将选定时间点对应的太阳光谱数据整合为二维矩阵，其中行表示波长采样点，列表示时间点，实现多时刻光谱的矩阵化存储。

Step 4 LED 通道光谱提取: 从 LED 光谱数据中提取预设的五个通道的光谱信息，构建多通道光谱矩阵，便于光谱通道间的定量比较与融合。

Step 5 光谱特征统计：对处理后的光谱数据计算关键统计指标，包括最大值、最小值、均值及总功率，全面描述光谱特征，为后续光谱合成、匹配及优化算法提供了数据基础和质量保障。

7.2 核心优化模型与求解方法

7.2.1 核心优化模型

基于第二问的结果，我们沿用 L-BFGS-B 算法，将 LED 光谱合成建模为一个带有边界约束的多目标优化问题，决策变量为全天各时间点五个通道的功率权重。优化目标是在满足物理约束的条件下，使合成光谱在 CCT 等五项指标上尽可能接近太阳光谱的目标轨迹，同时保持时序平滑性。

我们在目标函数中引入了以下两个约束：

多指标拟合项：将五个关键指标的误差按量纲进行归一化，并根据早、中、晚三个时段的特征分配不同的权重系数；

时序平滑项：通过一阶和二阶差分正则化限制相邻时间点及其变化趋势的幅度，从而获得平滑、连续的权重序列。

我们优化了目标函数，采用分时段加权的多指标跟踪损失与时序平滑正则化的组合形式，公示如下：

$$F(\mathbf{W}) = \sum_{t=1}^{T} \sum_{m \in \mathcal{M}} \alpha_{s(t),m} \cdot \left(\frac{I_m(\mathbf{W}t) - Im, \mathrm{sun}(t)}{\sigma_m} \right)^2 + \sum_{t=1}^{T-1} \beta_{s(t)} \cdot \|\mathbf{W}t + 1 - \mathbf{W}_t\|_2^2, \qquad (20)$$

其中，$\mathcal{M} = \{\text{CCT, Duv, mel-DER, Rf, Rg}\}$ 为五项关键生物节律指标集合，$I_m(\mathbf{W}t)$ 表示在权重 $\mathbf{W}_t$ 下合成光谱的第 m 项指标值，$Im, \text{sun}(t)$ 为目标太阳光谱的对应指标值，σ_m 为量纲归一化因子，$s(t) \in \{\text{morning, noon, evening}\}$ 为时间 t 对应的时段，$\alpha_{s(t),m}$ 和 $\beta_{s(t)}$ 分别为分时段的指标权重和平滑权重。

7.2.2 L-BFGS-B 算法的优势

L-BFGS-B 作为一种拟牛顿方法，通过有限内存存储 Hessian 矩阵的 BFGS 近似，在保证较快收敛的同时，显著减少了内存消耗。对于本研究的 75 维决策变量（15×5），能够高效利用目标函数的梯度信息，快速收敛。

L-BFGS-B 算法天然支持着边界约束处理，因此直接满足 LED 权重的物理边界条件 $0 \le W_{t,c} \le 1$，无需额外的约束处理机制，能够避免约束违反和数值不稳定问题。

该算法在 SciPy 优化库中已有成熟实现，避免了使用外部优化库，如 CVX 等，增强了代码的可移植性与稳定性。在实际测试中，L-BFGS-B 算法通常在 1000 次迭代内达到 10^{-9} 的收敛精度。

7.3 两阶段多目标时序优化策略

在问题三中，我们的目标是在全天光照变化过程中，满足多项生物节律与视觉舒适性指标的动态跟踪，同时兼顾光谱权重的时序平滑性。为此，我们设计了**两阶段多目标时序优化策略**，在保证关键指标拟合精度的同时，避免不必要的剧烈权重波动。

7.3.1 阶段 A：分时段全局拟合优化

阶段 A 的目标是在较大的参数范围内寻找一个整体表现稳定的初始解，为后续的精细优化打下基础。本阶段主要关注 CCT、Duv 和 mel-DER 这三项与节律密切相关的核心指标，在全天范围内让它们的变化趋势尽可能接近太阳光谱。

为了在保证精度的同时提升计算效率，我们采用了以下方式：

光谱重采样：将原始光谱以 5 nm 间隔重采样，由 401 个波长点压缩至 81 个点，计算量减少约 80%，在不明显影响拟合精度的前提下加快了优化过程。

分时段权重设置：早段（05:30–08:30）提高 mel-DER 权重，增强晨间节律激活；中段（09:30–15:30）保持三项指标的均衡；晚段（16:30–19:30）降低 mel-DER 权重并增加平滑约束，使光谱逐步过渡到暖色。

目标函数构建：在衡量合成光谱与目标光谱差异时，同时考虑三项核心指标的归一化误差，并加入时序平滑项限制相邻时间点权重的剧烈变化。公式如下：

$$F_A(\mathbf{W}) = \sum_{t=1}^{T} \left[\alpha_{\mathrm{CCT}} \left(\frac{\mathrm{CCT}(\mathbf{W}_t) - \mathrm{CCT}_{\mathrm{sun}}(t)}{\sigma_{\mathrm{CCT}}} \right)^2 + \alpha_{\mathrm{Duv}} \left(\frac{\mathrm{Duv}(\mathbf{W}_t) - \mathrm{Duv}_{\mathrm{sun}}(t)}{\sigma_{\mathrm{Duv}}} \right)^2 + \alpha_{\mathrm{mel}} \left(\frac{\text{mel-DER}(\mathbf{W}_t) - \text{mel-DER}_{\mathrm{sun}}(t)}{\sigma_{\mathrm{mel}}} \right)^2 \right] + \sum_{t=1}^{T-1} \beta \, \|\mathbf{W}_{t+1} - \mathbf{W}_t\|_2^2. \tag{21}$$

全局搜索求解：在 LED 各通道权重的 [0, 1] 边界条件下进行全局搜索，得到的结果作为阶段 B 的起点。

7.3.2 阶段 B：全时序平滑优化

在阶段 A 的结果基础上，阶段 B 进一步引入显色性相关的 2 个 TM-30 指标——Rf 与 Rg，对光谱进行精细化调整，使合成光谱在保持节律特性的同时具备更高的视觉品质。

稀疏计算策略：在优化过程中，仅在稀疏采样的时间点（每隔三个时间点取一次）计算 Rf 和 Rg 指标，并在目标函数中以全局权重 $w_{\mathrm{TM30}} = 0.3$ 进行加权。本策略能减少计算 TM-30 指标所需的大量运算，还保留对显色性能的有效约束。同时，在该阶段恢复使用完整 1 nm 分辨率的光谱数据，以确保显色性评估的精度。建模过程中该策略的使用使得优化过程通常在 300 次迭代内收敛，计算效率相比单阶段联合优化提升了约两倍。

目标函数构建：在阶段 A 目标函数的基础上，增加 Rf 与 Rg 两项归一化误差，并赋予全局 TM-30 权重 w_{TM30}。公式为：

$$F_B(\mathbf{W}) = F_A(\mathbf{W}) + w_{\mathrm{TM30}} \cdot \frac{1}{|\mathcal{T}_{\mathrm{sparse}}|} \sum_{t \in \mathcal{T}_{\mathrm{sparse}}} \left[\alpha_{R_f} \left(\frac{R_f(\mathbf{W}_t) - R_{f,\mathrm{sun}}(t)}{\sigma_{R_f}} \right)^2 + \alpha_{R_g} \left(\frac{R_g(\mathbf{W}_t) - R_{g,\mathrm{sun}}(t)}{\sigma_{R_g}} \right)^2 \right], \tag{22}$$

其中，$\mathcal{T}$sparse 为稀疏采样时间集合，Nsparse 为其元素数量，Nsparse 为稀疏时刻数量，$w_{\mathrm{TM30}} = 0.3$ 为 TM-30 指标的全局权重。

归一化与权重标定：为消除不同指标间的量纲差异，采用基于目标轨迹统计特性的自适应归一化策略：

$$\sigma_m = \max\{\mathrm{median}(|I_{m,\mathrm{sun}}|), 10^{-8}\}, \tag{23}$$

其中 σ_m 为指标 m 的归一化因子，median(·) 表示中位数运算。中位数的使用有效降低了异常值对归一化结果的影响。

权重参数的设定采用自动标定算法：首先计算太阳光谱在早、中、晚三时段的波动强度（标准差与梯度变化的加权组合），然后结合生物节律的生理学先验知识进行调制，得到适应性的分时段权重配置。

7.4 合成结果对比分析与性能评估

基于所提出的“两阶段光谱合成与节律优化”策略，模型在相关色温（CCT）、色度偏移（Duv）、褪黑素日光照度比（mel-DER）、显色指数 R_f 与色域指数 R_g 五个生物节律与光色质量关键指标上均实现了与目标太阳光谱的高精度轨迹跟踪，其中 CCT 与 mel-DER 的皮尔逊相关系数 r 均超过 0.99，表明拟合效果优异。皮尔逊相关系数的计算公式为：

$$r = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}, \tag{24}$$

其中 x_i、y_i 分别为目标与合成光谱在第 i 个时间点的指标值，$\bar{x}$、$\bar{y}$ 为其均值。

如图 13 所示，合成 LED 光源在五个指标的时间变化轨迹上与太阳光源高度一致，特别是在 CCT 与 mel-DER 两个与昼夜节律密切相关的指标上，误差波动幅度较小，确保了光源在动态调节过程中对人体生物钟的良好同步作用。

图 9　合成光谱与目标太阳光谱在五个关键指标上的轨迹对比

为进一步分析光谱合成精度，我们选取了早晨（05:30）、正午（12:30）、傍晚（19:30）

三个关键时间点，对比了合成光谱与目标太阳光谱的光谱功率分布（SPD）。光谱匹配度通过均方根误差（RMSE）进行量化：

$$\mathrm{RMSE} = \sqrt{\frac{1}{m}\sum_{j=1}^{m}\left[P_{\mathrm{LED}}(\lambda_j) - P_{\mathrm{Sun}}(\lambda_j)\right]^2}, \tag{25}$$

其中 $P_{\mathrm{LED}}(\lambda_j)$ 与 $P_{\mathrm{Sun}}(\lambda_j)$ 分别表示 LED 合成光谱与目标太阳光谱在波长 λ_j 处的功率值，m 为采样波长数。

图 10　05:30 时刻合成光谱与太阳光谱对比（RMSE = 0.6776）

图 11　12:30 时刻合成光谱与太阳光谱对比（RMSE = 0.7211）

图 12　19:30 时刻合成光谱与太阳光谱对比（RMSE = 0.5016）

结果显示，三时刻的 RMSE 分别为 0.6776、0.7211 和 0.5016，其中傍晚时段（19:30）的光谱拟合度最高，说明在低光照、暖色温条件下，LED 合成光谱更易精确还原太阳光

谱特征。这一特性对于夜间节律照明尤为重要，对短波蓝光的控制有助于降低对褪黑素分泌的抑制，促进夜间生理恢复。

综上所述，该两阶段优化方法在光色学指标上实现了与太阳光源的高拟合度，并且在生物节律关键参数上保持了高度同步，为可调控生物节律的 LED 光源设计提供了可靠的技术路径与量化依据。

八、 问题四：光照对睡眠质量影响的多条件比较与统计分析

8.1 睡眠数据预处理

Step 1: 将各受试者在不同光照条件下的原始睡眠记录进行格式统一，剔除记录不完整或存在明显测量误差的样本, 对于超出均值 $\pm 3\sigma$ 的极端数据剔除或修正。

Step 2: 对涉及时间的指标统一转换为分钟单位, 比例类指标均以百分数形式保留两位小数，对部分指标进行标准化处理。

Step 3: 将清洗后的数据按照实验分组（高褪黑素抑制光源、中性光源、低褪黑素抑制光源）进行重新排列，生成配对数据集，用于后续计算与配对检验。

8.2 睡眠指标算法

依据题目，我们的假设与六项指标的计算如下：

Part 1：设全夜睡眠记录共包含 N 个 epoch，每个 epoch 的时长为 $\Delta t = 30$ 秒。第 i 个 epoch 的睡眠阶段记为 S_i，其取值范围为 $\{4, 5, 2, 3\}$，分别对应清醒（Wake）、REM 睡眠、浅睡眠（N1+N2）与深睡眠（N3）。为便于计算，将所有睡眠阶段定义为集合

$$\mathcal{S} = \{2, 3, 5\}, \tag{26}$$

其中 $\mathcal{S}$ 用于表示非清醒状态（即处于浅睡眠、深睡眠或 REM 阶段）。

Part 2：总睡眠时间 (TST)：总睡眠时间指的是所有睡眠阶段所占的总时长，计算公式为：

$$\mathrm{TST} = \frac{\sum_{i=1}^{N} \mathbf{1}_{\{S_i \in \mathcal{S}\}} \times \Delta t}{60}, \tag{27}$$

其中，$\mathbf{1}_{\{S_i \in \mathcal{S}\}}$ 为指示函数，表示第 i 个 epoch 是否为睡眠阶段。将总时长单位转换为分钟。

Part 3：睡眠效率 (SE)：睡眠效率定义为总睡眠时间占总卧床时间的比例，计算为：

$$\mathrm{SE} = \frac{\mathrm{TST}}{\frac{N \times \Delta t}{60}} \times 100\%, \tag{28}$$

其中，分母为总卧床时间（即整个序列的时长，单位分钟）。

Part 4：入睡潜伏期 (SOL)：入睡潜伏期指从开始记录到第一次进入睡眠阶段的时间：

$$\mathrm{SOL} = \frac{\min\{i : S_i \in \mathcal{S}\} \times \Delta t}{60}. \tag{29}$$

若序列中没有任何睡眠阶段，则定义为总卧床时间：

$$\mathrm{SOL} = \frac{N \times \Delta t}{60}. \tag{30}$$

Part 5：深睡眠比例 (N3 percentage)：深睡眠比例是深睡眠阶段（N3）占总睡眠时间的百分比，计算为：

$$N3\% = \frac{\sum_{i=1}^{N} \mathbf{1}_{\{S_i=3\}}}{\sum_{i=1}^{N} \mathbf{1}_{\{S_i \in \mathcal{S}\}}} \times 100\%. \tag{31}$$

Part 6：REM 睡眠比例 (REM percentage)：REM 睡眠比例是 REM 阶段占总睡眠时间的百分比，计算为：

$$REM\% = \frac{\sum_{i=1}^{N} \mathbf{1}_{\{S_i=5\}}}{\sum_{i=1}^{N} \mathbf{1}_{\{S_i \in \mathcal{S}\}}} \times 100\%. \tag{32}$$

Part 7：夜间醒来次数 (Number of Awakenings)：夜间醒来次数定义为睡眠开始后，每次由睡眠阶段转变为清醒阶段的次数。令醒来计数器 $A = 0$，状态变量 $sleep_started =$ False，遍历序列时：

$$\text{for } i = 1 \text{ to } N: \quad \begin{cases} \text{若} S_i \in \mathcal{S}, & sleep_started = \text{True} \\ \text{若 sleep_started = True 且} & \\ \quad S_i = 4, A = A + 1, & sleep_started = \text{False}, \end{cases} \tag{33}$$

即每当睡眠状态开始后出现清醒状态，计为一次醒来。

8.3 统计分析

为全面评估不同光环境对睡眠质量的影响，我们在统计分析过程中引入了由基础分布检验到重复测量比较的多层次方法体系，使结果既具统计学稳健性，又能反映实际生理意义。

8.3.1 描述性统计

在推断性分析之前，我们先对三种光照条件下的六项核心睡眠质量指标进行描述性统，计算每组的样本数量、均值、标准差、最小值和最大值，用于直观比较不同条件下的集中趋势和波动情况，也为后续的统计检验提供分布参考。

8.3.2 分布与方差假设检验

进行以下检验的原因是本实验涉及同一批受试者在三种光环境下的重复测量，参数分析方法要求数据满足正态性与方差齐性，否则需考虑非参数替代方法。

正态性检验 采用 Shapiro–Wilk 检验评估各指标在每种光照条件下的分布是否近似正态，公示如下：

$$W = \frac{\left(\sum_{i=1}^{n} a_i x_{(i)}\right)^2}{\sum_{i=1}^{n}(x_i - \bar{x})^2} \tag{34}$$

其中，$x_{(i)}$ 为按大小排序的样本值，a_i 为基于协方差矩阵计算得到的常数。若 $p > 0.05$，可认为数据分布满足正态性假设。

方差齐性检验 采用 Levene 检验判断三组数据的方差是否相等：

$$W = \frac{(N-k)}{(k-1)} \cdot \frac{\sum_{j=1}^{k} n_j (Z_{\cdot j} - Z_{\cdot\cdot})^2}{\sum_{j=1}^{k}\sum_{i=1}^{n_j}(Z_{ij} - Z_{\cdot j})^2} \tag{35}$$

其中，$Z_{ij} = |Y_{ij} - \tilde{Y}_j|$，$\tilde{Y}_j$ 为第 j 组的中位数。若 $p > 0.05$，视为方差齐性成立。

8.3.3 重复测量方差分析与事后比较

由于所有受试者均在三种光环境条件（助眠模式光谱、普通市售 LED 灯、严格黑暗）下分别接受测试，且记录了多个睡眠质量指标，因此数据具有“同一对象多次测量”的重复性结构。与独立样本 ANOVA 相比，单因素重复测量 ANOVA 能有效消除个体间差异对统计结果的干扰，从而更敏感地检出条件间的真实差异。

在分析中，将“光环境条件”作为自变量，各睡眠质量指标（总睡眠时间、睡眠效率、入睡潜伏期、深睡眠比例、REM 睡眠比例、夜间醒来次数）作为因变量。ANOVA 的统计量计算公式为：

$$F = \frac{\text{条件效应的均方}}{\text{误差均方}},$$

显著性水平设定为 $\alpha = 0.05$。当 $p < 0.05$ 时，判定不同光环境条件下该指标的均值存在显著差异。

然而，ANOVA 仅能判断总体差异是否存在，并不能直接揭示差异来源。因此，对显著或边缘显著的指标进一步进行三组条件的两两配对 t 检验（助眠模式 vs 普通 LED、助眠模式 vs 黑暗、普通 LED vs 黑暗）。配对 t 检验能够利用被试内差异信息，对两条件间的均值差进行显著性判断，其计算公式为：

$$t = \frac{\bar{d}}{s_d/\sqrt{n}},$$

其中 $\bar{d}$ 为两条件下的均值差，s_d 为该差值的标准差，n 为样本量。为避免多重比较导致的 I 型错误膨胀，采用 Bonferroni 校正调整显著性水平。

此外，为评估差异的实际影响大小，计算 Cohen’s d 效应量：

$$d = \frac{\bar{X}_1 - \bar{X}_2}{s_p}, \quad s_p = \sqrt{\frac{s_1^2 + s_2^2}{2}},$$

其中 s_p 为两组的合并标准差。效应量解释标准为：$d \approx 0.2$（小效应）、$d \approx 0.5$（中效应）、$d \approx 0.8$（大效应）。

$$F = \frac{\text{条件效应的均方}}{\text{误差均方}},$$

显著性水平设定为 $\alpha = 0.05$。当 $p < 0.05$ 时，表明不同条件下的该指标存在显著差异。

为进一步确定差异来源，对显著或边缘显著的指标进行三组两两配对 t 检验（助眠模式 vs 普通 LED、助眠模式 vs 黑暗、普通 LED vs 黑暗）。配对 t 检验公式为：

$$t = \frac{\bar{d}}{s_d/\sqrt{n}},$$

并采用 Bonferroni 校正控制多重比较带来的 I 型错误风险。

此外，计算 Cohen’s d 效应量：

$$d = \frac{\bar{X}_1 - \bar{X}_2}{s_p}, \quad s_p = \sqrt{\frac{s_1^2 + s_2^2}{2}},$$

用于评估差异的实际意义大小（$d \approx 0.2$ 为小效应，$d \approx 0.5$ 为中效应，$d \approx 0.8$ 为大效应）。

8.4 结果验证

表 7　三种光照条件下睡眠指标的描述性统计

睡眠指标	夜间助眠模式	普通 LED 灯	严格黑暗环境
总睡眠时间（分钟）	366.86±49.7	379.14±58.96	359.68±53.3
睡眠效率（%）	85.72±8.07	87.48±9.29	88.36±8.07
入睡潜伏期（分钟）	20.14±12.08	21.73±24.56	10.41±16.05
深睡眠比例（%）	19.01±4.4	19.39±7.12	24.66±4.95
REM 睡眠比例（%）	25.28±7.07	26.82±7.46	21.2±7.35
夜间醒来次数	14.73±3.47	14.18±4.73	13.82±4.81

图 13　三条件六指标箱线图

描述性统计结果如表 7 及图 13 所示，表内的数据为均值加减方差。在总睡眠时间方面，夜间助眠模式组略低于普通 LED 灯组，高于严格黑暗环境组。睡眠效率在助眠模式下为 85.72±8.07%，略低于普通 LED 灯组和黑暗环境组。入睡潜伏期方面，助眠模式组较普通 LED 灯组缩短约 1.6 分钟，但较黑暗环境组延长约 9.7 分钟。

在深睡眠比例方面，助眠模式组为 19.01±4.40%，为三组中最低，而黑暗环境组最高，普通 LED 灯组处于中间水平。夜间助眠模式在 REM 睡眠比例处于三组的中间水平，普通 LED 灯组略高，黑暗环境组最低。夜间助眠模式在夜间醒来次数方面为 14.73±3.47 次，略高于普通 LED 灯组和黑暗环境组。

表 8　重复测量 ANOVA 结果

睡眠指标	F 统计量	p 值	显著性
总睡眠时间	1.21	0.312	无显著差异
睡眠效率	0.89	0.424	无显著差异
入睡潜伏期	1.56	0.228	无显著差异
深睡眠比例	3.48	0.0438	显著差异
REM 睡眠比例	2.15	0.135	无显著差异
夜间醒来次数	0.23	0.798	无显著差异

如表 8 所示，六个睡眠指标中仅深睡眠比例在三种条件间存在显著差异（F=3.48，p=0.0438）；其余指标差异均未达到显著水平（p>0.05）。

表 9　深睡眠比例的配对比较结果

比较组合	t 统计量	自由度	p 值	Cohen’s d	显著性
助眠模式 vs 黑暗环境	-3.22	10	0.009	-1.24	高度显著
普通 LED vs 黑暗环境	-2.10	10	0.062	-0.78	边缘显著
助眠模式 vs 普通 LED	-0.14	10	0.894	-0.06	无显著差异

如表 9 所示，进一步的事后配对 t 检验发现，深睡眠比例在助眠模式与黑暗环境间差异显著（p=0.009，Cohen’s d=-1.24），普通 LED 与黑暗环境的差异仅达到边缘显著水平（p=0.062，Cohen’s d=-0.78），助眠模式与普通 LED 间无显著差异（p=0.894）。

表 10　夜间助眠模式的方向性改善分析

睡眠指标	预期方向	vs 普通 LED	vs 黑暗环境	整体评估
总睡眠时间	增加	-12.3 分钟	+7.2 分钟	混合效果
睡眠效率	提高	-1.8%	-2.6%	无改善
入睡潜伏期	减少	-1.6 分钟	+9.7 分钟	混合效果
深睡眠比例	增加	-0.4%	-5.7%	显著较差
REM 睡眠比例	优化	-1.5%	+4.1%	混合效果
夜间醒来次数	减少	+0.6 次	+0.9 次	无改善

结合方向性改善分析（如表 10 所示），助眠模式在部分指标上相较对照条件呈现改善趋势，例如入睡潜伏期较普通 LED 缩短、REM 比例较黑暗环境增加。但在深睡眠比例这一与睡眠恢复质量高度相关的指标上，助眠模式显著低于黑暗环境，并在总睡眠时间、睡眠效率和夜间醒来次数等指标上未表现出稳定优势。

整体来看，本次实验的助眠光照方案在部分睡眠结构指标上有一定的优化表现，但未能在整体睡眠质量上呈现统计学上的全面优势，且在深睡眠比例上存在明显不足。这提示在后续方案优化中，需要进一步调整光谱参数与照度水平，以在缩短入睡时间的同时，兼顾深睡眠比例和睡眠效率的提升。

九、 模型评价

9.1 模型优点

为验证不同环节模型的适用性与稳定性，本文在光谱拟合、权重优化和实验验证三个阶段均设置了对比与评估指标。整体方法的优点主要包括：

- **理论与标准结合**：所有光谱评价指标均基于 CIE S 026/E:2018 与 IES TM-30-20 标准计算，确保结果的国际可比性；
- **多目标优化策略**：在权重寻优中引入 SLSQP、L-BFGS-B、差分进化、盆地跳跃、双退火等多种算法并行计算，提高了高维复杂约束下的收敛稳定性；
- **分阶段优化框架**：采用“节律优先—显色性补偿—工程可行性约束”的三阶段优化策略，使光谱在多维指标上均衡提升；

- **实验闭环验证**：通过睡眠实验将优化光谱与普通 LED、黑暗环境进行对照，确保优化效果在实际场景下具有可观测的生理反应。

9.2 模型不足

尽管整体框架在理论建模与数值优化上具有稳定表现，但在实验环节仍存在部分局限：

- 样本量有限（$n = 11$），导致部分指标虽有改善趋势但未达到显著性水平；
- 实验时长较短，未能评估光谱长期作用下的节律与睡眠变化；
- 模型未考虑个体差异，后续可结合个体光照响应模型进行个性化优化。

9.3 模型稳定性与适用性

为检验模型在不同场景下的稳定性，分别在场景一、场景二以及三阶段优化中记录了关键指标变化的均值与方差。结果表明：

- 在权重优化阶段，不同初值下的多次迭代结果收敛至相近最优值，标准差均低于 5%；
- 在光谱拟合阶段，节律性指标（mel-DER）的拟合误差均方根（RMSE）控制在 0.05 以内；
- 在实验验证阶段，深睡眠比例在助眠模式与黑暗环境对比中达到了显著差异（$p = 0.009$），表明节律性调控在生理层面具有可检测的效应。

参考文献

[1] Michel Gendreau and Jean-Yves Potvin, editors. Handbook of Metaheuristics. Springer, 3rd edition, 2019.

[2] Jorge Nocedal and Stephen J. Wright. Numerical Optimization. Springer, 2nd edition, 2006.

[3] Michael P. Royer. Tutorial: Background and guidance for using the ansi/ies tm-30 method for evaluating light source color rendition. Leukos, 18(2):191–231, 2022.

[4] Luc J. M. Schlangen, Luke Price, David Sliney, and Manuel Spitschan. Report on the workshop use and application of the new cie s 026/e:2018, metrology for iprgc-influenced responses to light “specifying light for its eye-mediated non-visual effects in humans”. In Proceedings of the 29th Session of the CIE, volume 2, pages 114–118. CIE, 2019.

[5] 张浩，徐海松. 光源相关色温算法的比较研究. 光学仪器, (01):54–58, 2006.

[6] 李宁. LED 混光的光视效能及显色性评估研究. 博士学位论文, 北京大学, 2014.

[7] 李月. 光源相关色温及色偏差计算方法研究. 硕士学位论文, 辽宁科技大学, 2023.

[8] 蔡连章， 张国芳，潘利兵. 通用照明白光 led 器件光谱节律刺激效应分析. 照明工程学报, 31(05):63–66, 2020.

```
  def read_multi_channel_spd(xlsx_path):

    df = pd.read_excel(xlsx_path)
    if df.shape[1] < 2:
        raise ValueError('appendix#2.xlsx 至少需要2列（波长 + 至少1个通道）')


    wl_series = df.iloc[:, 0].astype(str)
    wl_list = []
    for s in wl_series:
        s = '' if s is None else s
        import re
        nums = re.findall(r"\d+", s)
        if nums:
            wl_list.append(int(nums[0]))
        else:
            wl_list.append(np.nan)
    wavelengths_raw = pd.to_numeric(pd.Series(wl_list), errors='coerce').values
    valid = ~np.isnan(wavelengths_raw)
    wavelengths_raw = wavelengths_raw[valid]

    channel_names = [str(c) for c in df.columns[1:]]
    channel_spds = {}
    for idx, cname in enumerate(channel_names, start=1):
        spd_vals = pd.to_numeric(df.iloc[:, idx], errors='coerce').values
        spd_vals = spd_vals[valid]
        spd_vals = np.nan_to_num(spd_vals, nan=0.0)
        spd_vals = np.maximum(spd_vals, 0.0)
        channel_spds[cname] = spd_vals

    return wavelengths_raw, channel_names, channel_spds


def make_uniform_grid(wl_min=380, wl_max=780, step=5):
    return np.arange(wl_min, wl_max + 1e-9, step)


def resample_to_grid(wavelengths_raw, spd_raw, grid):

    order = np.argsort(wavelengths_raw)
    wl = np.asarray(wavelengths_raw)[order]
    spd = np.asarray(spd_raw)[order]
```

```
    uniq_wl, idx = np.unique(wl, return_index=True)
    uniq_spd = spd[idx]

    spd_interp = np.interp(grid, uniq_wl, uniq_spd, left=0.0, right=0.0)
    return spd_interp


def normalize_spd(spd):
    m = np.max(spd) if np.max(spd) > 0 else 1.0
    return spd / m


class MultiChannelMixer:
    def __init__(self, xlsx_path, wl_grid=None):
        self.xlsx_path = xlsx_path
        self.wl_grid = wl_grid if wl_grid is not None else make_uniform_grid()
        self.wavelengths_raw, self.channel_names, self.channel_spds_raw =
            read_multi_channel_spd(xlsx_path)


        self.channel_spds = {}
        for name in self.channel_names:
            spd_i = resample_to_grid(self.wavelengths_raw, self.channel_spds_raw[name],
                self.wl_grid)

            self.channel_spds[name] = normalize_spd(spd_i)

        self.num_channels = len(self.channel_names)


        print("初始化TM-30优化计算器...")
        self.tm30_calc = TM30CalculatorOptimized(self.wl_grid, self.channel_spds, verbose=False)
        print("TM-30预计算完成! ")

    def combine(self, weights):
        w = np.asarray(weights).reshape(-1)
        if len(w) != self.num_channels:
            raise ValueError('权重长度与通道数不一致')

        spd = np.zeros_like(self.wl_grid, dtype=float)
        for i, name in enumerate(self.channel_names):
            spd += w[i] * self.channel_spds[name]
```

```
        return normalize_spd(spd)

    def compute_cct(self, spd):

        X, Y, Z = cctcalc.calculate_xyz_from_spd(self.wl_grid, spd)
        _, _, u, v = cctcalc.calculate_chromaticity_coordinates(X, Y, Z)
        cct_val, _info = calculate_cct_triangle_detailed(u, v, temp_step=50)
        return float(cct_val)

    def compute_tm30(self, spd):

        raise NotImplementedError("请使用 compute_tm30_from_weights 方法")

    def compute_tm30_from_weights(self, weights):

        return self._cached_tm30_compute(tuple(np.round(weights, 6)))

    @lru_cache(maxsize=1000)
    def _cached_tm30_compute(self, weights_tuple):

        weights = np.array(weights_tuple)
        res = self.tm30_calc.calculate_tm30_from_weights(weights)
        rf = float(res['rf_overall'])
        rg = float(res['rg_overall'])
        cct_tm = float(res['cct_test'])
        return rf, rg, cct_tm

    def clear_tm30_cache(self):

        self._cached_tm30_compute.cache_clear()


def optimize_weights_global(mixer, target_cct=6000.0, cct_tol=500.0, rg_range=(95.0, 105.0),
    rf_min=88.0, max_iter=200, early_stop_rf=None):

    n = mixer.num_channels
    if minimize is None:
        raise RuntimeError('scipy.optimize 未安装，无法执行优化')

    bounds = [(0.0, 1.0)] * n


    global_progress = {"iter": 0, "last_metrics": None}
```

```
def penalty_objective(w):
    if np.sum(w) <= 0:
        return 1e6


    w_norm = w / np.sum(w)

    try:
        rf, rg, _ = mixer.compute_tm30_from_weights(w_norm)
        spd = mixer.combine(w_norm)
        cct_val = mixer.compute_cct(spd)


        global_progress["iter"] += 1
        global_progress["last_metrics"] = (rf, rg, cct_val)


        obj = 100.0 - rf


        penalty = 0.0


        cct_violation = 0
        if cct_val < target_cct - cct_tol:
            cct_violation = (target_cct - cct_tol - cct_val) / 100
        elif cct_val > target_cct + cct_tol:
            cct_violation = (cct_val - target_cct - cct_tol) / 100
        penalty += 500 * cct_violation ** 2

        # Rg约束
        if rg < rg_range[0]:
            penalty += 100 * (rg_range[0] - rg) ** 2
        elif rg > rg_range[1]:
            penalty += 100 * (rg - rg_range[1]) ** 2

        # Rf约束
        if rf < rf_min:
            penalty += 200 * (rf_min - rf) ** 2

        return obj + penalty

    except Exception as e:
```

```
        return 1e6

# 定义约束（用于局部算法）
def c_ct_upper(w):
    spd = mixer.combine(w)
    cct_val = mixer.compute_cct(spd)
    return (target_cct + cct_tol) - cct_val

def c_ct_lower(w):
    spd = mixer.combine(w)
    cct_val = mixer.compute_cct(spd)
    return cct_val - (target_cct - cct_tol)

def c_rg_upper(w):
    rf, rg, _ = mixer.compute_tm30_from_weights(w)
    return rg_range[1] - rg

def c_rg_lower(w):
    rf, rg, _ = mixer.compute_tm30_from_weights(w)
    return rg - rg_range[0]

def c_rf_min(w):
    rf, rg, _ = mixer.compute_tm30_from_weights(w)
    return rf - rf_min

def objective_local(w):
    rf, rg, _ = mixer.compute_tm30_from_weights(w)
    return 100.0 - rf

constraints = [
    {"type": "eq", "fun": lambda w: np.sum(w) - 1.0},
    {"type": "ineq", "fun": c_ct_upper},
    {"type": "ineq", "fun": c_ct_lower},
    {"type": "ineq", "fun": c_rg_upper},
    {"type": "ineq", "fun": c_rg_lower},
    {"type": "ineq", "fun": c_rf_min},
]

# 生成高质量初值
def generate_smart_initials():
    initials = []

    # 1. 均分
    eq = np.ones(n) / n
```

```
    initials.append(eq)

    # 2. 单通道主导
    for i in range(n):
        v = np.ones(n) * 0.1 / (n - 1)
        v[i] = 0.9
        initials.append(v)

    # 3. 双通道组合
    for i in range(n-1):
        v = np.ones(n) * 0.05 / (n - 2)
        v[i] = 0.475
        v[i+1] = 0.475
        initials.append(v)

    # 4. 随机高质量初值
    np.random.seed(42)
    for _ in range(8):
        w = np.random.dirichlet(np.ones(n))
        initials.append(w)

    return initials

# 通用进度回调函数（用于全局算法）
def create_global_callback(alg_name, expected_calls):
    """创建带进度条的回调函数"""
    progress_bar = None
    if tqdm is not None:
        progress_bar = tqdm(total=expected_calls, desc=f"{alg_name}", position=1,
            leave=False)

    callback_counter = {"count": 0}

    def callback(xk, convergence=None):
        nonlocal progress_bar
        callback_counter["count"] += 1

        if progress_bar is not None:
            progress_bar.update(1)
            # 每5次回调更新一次详细信息，减少计算开销
            if callback_counter["count"] % 5 == 0 and global_progress["last_metrics"]:
                rf, rg, cct_val = global_progress["last_metrics"]
                progress_bar.set_postfix({
                    "Rf": f"{rf:.1f}",
```

```
                    "Rg": f"{rg:.1f}",
                    "CCT": f"{cct_val:.0f}K",
                    "Calls": global_progress["iter"]
                })
            elif callback_counter["count"] % 5 != 0:
                # 非更新周期只显示基本信息
                progress_bar.set_postfix({
                    "Calls": global_progress["iter"]
                })
        else:
            # 简易文本进度（减少输出频率）
            if global_progress["iter"] % 100 == 0 and global_progress["last_metrics"]:
                rf, rg, cct_val = global_progress["last_metrics"]
                print(f" {alg_name} | 调用={global_progress['iter']}: Rf={rf:.1f},
                    Rg={rg:.1f}, CCT={cct_val:.0f}K")

    def cleanup():
        if progress_bar is not None:
            progress_bar.close()

    return callback, cleanup
```

附录 B 合成光谱的两阶段优化

```
def _stage_a_objective(self, weights_flat: np.ndarray, target_trajectory: Dict[str,
    np.ndarray]) -> float:
    """阶段A目标函数: mel-DER + CCT + Duv + 平滑"""
    W = weights_flat.reshape(self.n_times, self.n_channels)

    total_loss = 0.0


    if self.config.enable_5nm_speedup:
        wavelengths = self.wavelengths_5nm
        led_spectra = self.led_spectra_5nm
    else:
        wavelengths = self.wavelengths
        led_spectra = self.led_spectra

    # 逐时刻指标损失
    for t in range(self.n_times):
        segment = self._get_segment_for_time(t)
```

```
        if segment not in self.segment_weights:
            continue

        sw = self.segment_weights[segment]

        # 合成光谱
        synth_spd = led_spectra @ W[t, :]

        # 计算关键指标（阶段A只需CCT/Duv/mel-DER）
        try:
            metrics = compute_all_metrics(wavelengths, synth_spd, self.metrics_cache)

            # 归一化误差
            cct_err = ((metrics['CCT'] - target_trajectory['CCT'][t]) /
                self.scale_factors['CCT']) ** 2
            duv_err = ((metrics['Duv'] - target_trajectory['Duv'][t]) /
                self.scale_factors['Duv']) ** 2
            mel_err = ((metrics['mel_DER'] - target_trajectory['mel_DER'][t]) /
                self.scale_factors['mel_DER']) ** 2

            # 加权损失
            total_loss += sw.cct * cct_err + sw.duv * duv_err + sw.mel_der * mel_err

        except Exception as e:
            # 数值计算失败时的惩罚
            total_loss += 1000.0

    # 时序平滑项
    for t in range(self.n_times - 1):
        segment = self._get_segment_for_time(t)
        if segment not in self.segment_weights:
            continue

        sw = self.segment_weights[segment]

        # 一阶平滑
        diff1 = W[t+1, :] - W[t, :]
        total_loss += sw.temporal_smooth_l1 * np.sum(diff1 ** 2)

    # 二阶平滑（可选）
    if hasattr(self.segment_weights[list(self.segment_weights.keys())[0]],
        'temporal_smooth_l2'):
        for t in range(1, self.n_times - 1):
            segment = self._get_segment_for_time(t)
```

```
            if segment not in self.segment_weights:
                continue

            sw = self.segment_weights[segment]

            # 二阶平滑
            diff2 = W[t+1, :] - 2*W[t, :] + W[t-1, :]
            total_loss += sw.temporal_smooth_l2 * np.sum(diff2 ** 2)

    return float(total_loss)

def _stage_b_objective(self, weights_flat: np.ndarray, target_trajectory: Dict[str,
    np.ndarray]) -> float:
    """阶段B目标函数: 在A基础上加入稀疏Rf/Rg"""
    # 首先计算阶段A的损失
    loss_a = self._stage_a_objective(weights_flat, target_trajectory)

    W = weights_flat.reshape(self.n_times, self.n_channels)

    # 稀疏Rf/Rg损失
    rf_rg_loss = 0.0
    rf_rg_count = 0

    for t in range(0, self.n_times, self.config.stage_b_sparse_interval):
        segment = self._get_segment_for_time(t)
        if segment not in self.segment_weights:
            continue

        sw = self.segment_weights[segment]

        # 合成光谱 (使用完整1nm数据)
        synth_spd = self.led_spectra @ W[t, :]

        try:
            metrics = compute_all_metrics(self.wavelengths, synth_spd, self.metrics_cache)

            # Rf/Rg误差
            rf_err = ((metrics['Rf'] - target_trajectory['Rf'][t]) /
                self.scale_factors['Rf']) ** 2
            rg_err = ((metrics['Rg'] - target_trajectory['Rg'][t]) /
                self.scale_factors['Rg']) ** 2

            rf_rg_loss += sw.rf * rf_err + sw.rg * rg_err
            rf_rg_count += 1
```

```
            except Exception:
                rf_rg_loss += 100.0

        # 归一化Rf/Rg损失
        if rf_rg_count > 0:
            rf_rg_loss /= rf_rg_count

        # 总损失
        total_loss = loss_a + self.config.stage_b_rf_rg_weight * rf_rg_loss

        return float(total_loss)

    def optimize_stage_a(self, target_trajectory: Dict[str, np.ndarray]) -> Tuple[np.ndarray,
        dict]:

        print("开始阶段A优化 (mel-DER + CCT + Duv + 平滑)...")


        self._setup_progress_bar("阶段A", self.config.stage_a_max_iter)

        # 初始权重
        W0 = np.full((self.n_times, self.n_channels), 0.15)
        weights_flat = W0.flatten()

        # 边界约束
        bounds = [self.config.weight_bounds] * len(weights_flat)

        # 优化
        start_time = time.time()

        result = minimize(
            fun=self._stage_a_objective,
            x0=weights_flat,
            args=(target_trajectory,),
            method='L-BFGS-B',
            bounds=bounds,
            callback=self._progress_callback,
            options={
                'maxiter': self.config.stage_a_max_iter,
                'ftol': self.config.stage_a_ftol
            }
        )
```

```
        stage_a_time = time.time() - start_time


        self._close_progress_bar()

        # 重塑权重
        W_opt_a = result.x.reshape(self.n_times, self.n_channels)

        stage_a_info = {
            'success': result.success,
            'message': result.message,
            'final_loss': result.fun,
            'iterations': result.nit,
            'time': stage_a_time,
            'weights': W_opt_a
        }

        print(f"阶段A完成: {'成功' if result.success else '未完全收敛'}")


        return W_opt_a, stage_a_info

    def optimize_stage_b(self, W_init: np.ndarray, target_trajectory: Dict[str, np.ndarray]) ->
        Tuple[np.ndarray, dict]:

        print("开始阶段B优化 (加入Rf/Rg精修)...")


        self._setup_progress_bar("阶段B", self.config.stage_b_max_iter)

        # 从阶段A结果开始
        weights_flat = W_init.flatten()

        # 边界约束
        bounds = [self.config.weight_bounds] * len(weights_flat)

        # 优化
        start_time = time.time()

        result = minimize(
            fun=self._stage_b_objective,
            x0=weights_flat,
            args=(target_trajectory,),
            method='L-BFGS-B',
```

```
            bounds=bounds,
            callback=self._progress_callback,
            options={
                'maxiter': self.config.stage_b_max_iter,
                'ftol': self.config.stage_b_ftol
            }
        )

        stage_b_time = time.time() - start_time


        self._close_progress_bar()

        # 重塑权重
        W_opt_b = result.x.reshape(self.n_times, self.n_channels)

        stage_b_info = {
            'success': result.success,
            'message': result.message,
            'final_loss': result.fun,
            'iterations': result.nit,
            'time': stage_b_time,
            'weights': W_opt_b
        }

        print(f"阶段B完成: {'成功' if result.success else '未完全收敛'}")

        return W_opt_b, stage_b_info

    def optimize(self, sun_spectra: np.ndarray, weights_config_path: str) -> Dict:

        print("=" * 60)
        print("两阶段节律优化")
        print("=" * 60)

        start_time = time.time()

        # 1. 加载配置
        self.load_segment_weights(weights_config_path)

        # 2. 计算目标轨迹
        print("计算太阳光谱目标轨迹...")
        target_trajectory = compute_target_trajectory(sun_spectra, self.wavelengths,
            self.metrics_cache)
```

```
        # 3. 估计缩放因子
        self._fit_scale_factors(target_trajectory)

        # 4. 阶段A优化
        W_stage_a, info_a = self.optimize_stage_a(target_trajectory)

        # 5. 阶段B优化
        W_stage_b, info_b = self.optimize_stage_b(W_stage_a, target_trajectory)

        total_time = time.time() - start_time

        # 6. 构建结果
        results = {
            'weights_final': W_stage_b,
            'weights_stage_a': W_stage_a,
            'target_trajectory': target_trajectory,
            'stage_a_info': info_a,
            'stage_b_info': info_b,
            'total_time': total_time,
            'config': self.config,
            'scale_factors': self.scale_factors,
            'segment_weights': self.segment_weights
        }
```